

Week 1

Classificatie met Naive Bayes, KNN

Herhalen

Wat weten jullie hier nog over ?

- Classificatie vs Clustering
- Supervised (begeleid) vs Unsupervised (onbegeleid)
- Underfitting & Overfitting
- Train / Test / Validation data
- Cross-validatie

flowchart LR

Data["Dataset"] → Split["Split 80/20"]

Split → Train["Train set"]

Split → Test["Test set"]

Train → CV["Cross-validatie
(bv. 5-fold)"]

CV → Model["Getraind model"]

Model → Eval["Evalueren op test set"]

Naive Bayes

Een eerste classificer

Conditionele kans (voorwaardelijke kans)

- $P(A \mid B)$: kans op A als B al gebeurd is
- Definitie:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

- Voorbeeld — Dobbelsteen:
 - $P(\text{twee keer zes}) = 1/36 = P(A \cap B)$
 - $P(\text{é é n keer zes}) = 1/6 = P(B)$
 - $P(\text{twee keer zes} \mid \text{al é é n keer zes}) = \frac{1/36}{1/6} = \frac{1}{6}$

Regel van Bayes — formule

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B)}$$

Alternatieve vorm (handig in de praktijk):

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B \mid A)P(A) + P(B \mid \neg A)P(\neg A)}$$

K MERMAID DIAGRAM — Bayes Rule flow

```
flowchart LR
    A["P(B|A)"] --> C["P(B|A) · P(A)"]
    B["P(A)"] --> C
    C --> D["P(A|B)"]
    E["P(B)"] --> D
```

Regel van Bayes — in ML-context

- **B** = data / feiten (de e-mail)
- **A** = het classificatielabel dat we willen toekennen (spam / ham)

We willen dus berekenen:

$$P(\text{label} \mid \text{data})$$

*"Wat is de kans op dit label, **gegeven** de data die we zien?"*

Spamfilter — probleemstelling

We bouwen een **spamfilter** dat e-mails labelt als 'spam' of 'geen spam'.

Gegeven:

- $P(\text{moneytransfer} \mid \text{spam}) = 0.4$ (40% van spam bevat 'moneytransfer')
- $P(\text{moneytransfer} \mid \neg \text{spam}) = 0.0001$ (0.01% van ham bevat het)
- $P(\text{spam}) = 0.45$ (45% van alle e-mail is spam)

Vraag: Een e-mail bevat 'moneytransfer' — is het spam?

flowchart TD

```
Email["Email ontvangen"] → Check{"Bevat 'moneytransfer'?"}  
Check -- Ja → Bereken["Bereken  $P(\text{spam}|\text{mt})$  en  $P(\neg\text{spam}|\text{mt})$ "]  
Check -- Nee → Next["Check volgende woorden"]  
Bereken → Kies{"Welke kans is groter?"}  
Kies -- Spam → LabelS[" SPAM"]  
Kies -- Ham → LabelH[" HAM"]
```

Moneytransfer → Spam?

Berekening via Bayes:

$$\begin{aligned} P(\text{spam} \mid \text{mt}) &= \frac{P(\text{mt} \mid \text{spam}) P(\text{spam})}{P(\text{mt} \mid \text{spam}) P(\text{spam}) + P(\text{mt} \mid \neg \text{spam}) P(\neg \text{spam})} \\ &= \frac{0.4 \times 0.45}{0.4 \times 0.45 + 0.0001 \times 0.55} \\ &\approx 0.9997 \end{aligned}$$

Besluit: $P(\text{spam} \mid \text{mt}) \approx 99.97\% \rightarrow \text{label} = \text{spam}$ 

Meerdere woorden screenen — gegevens

'moneytransfer'	0.4	0.0001
'uganda'	0.1	0.05
'AP Hogeschool'	0.001	0.3

Vraag: Een e-mail bevat **niet** 'moneytransfer', maar **wel** 'uganda' en 'AP Hogeschool'.
Wat is de kans op spam?

Meerdere datapunten — berekening

Naieve aanname: woorden zijn onafhankelijk $\rightarrow$ product van kansen

Voor spam:

$$P(\neg \text{mt} \mid \text{spam}) \times P(\text{ug} \mid \text{spam}) \times P(\text{ap} \mid \text{spam}) \times P(\text{spam})$$

Voor geen spam:

$$P(\neg \text{mt} \mid \neg \text{spam}) \times P(\text{ug} \mid \neg \text{spam}) \times P(\text{ap} \mid \neg \text{spam}) \times P(\neg \text{spam})$$

Uiteindelijk: kies het label met ***grootste teller*** — de noemer $P(\text{data})$ is voor beide gelijk, dus hoeft je niet uit te rekenen!

```
flowchart LR
    subgraph Werkelijkheid["Werkelijkheid ✗"]
        W1["moneytransfer ↔ uganda<br/>woorden hangen samen"]
    end
    subgraph Aannname["Naieve aannname ✔"]
        A1["moneytransfer ⊥ uganda<br/>onafhankelijk behandeld"]
    end
```

Naive Bayes Classifier

Waarom "naief"?

- De kansen van woorden **samen** en **apart** zijn niet onafhankelijk — in werkelijkheid beïnvloeden ze elkaar.
- Toch werkt deze classifier **in de praktijk verrassend goed**, dus we negeren dit bezwaar.

Laplace-smoothing

Als $P(x_i \mid y = k) = 0$ voor een label, wordt het hele product 0.

Oplossing: tel een kleine waarde δ op:

$$P(x_i \mid y = k) = \frac{\delta + \#(x_i, y = k)}{\sum_j (\delta + \#(x_j, y = k))}$$

- δ wordt bepaald via cross-validatie.

Implementatie in sklearn

```
from sklearn.naive_bayes import BernoulliNB  
clf = BernoulliNB()  
clf.fit(X, Y)  
clf.predict(X[2:3])
```

*Volgende stap: het trainen van de data flexibeler maken → **Labo!***

K-Nearest Neighbours (KNN)

Wat bepaalt de prijs van een huis?

Locatie, locatie en nog eens locatie!

flowchart LR

Huis["🏠 Nieuw huis
(prijs = ?)"] → B1["📍 Buur 1: €350k"]

Huis → B2["📍 Buur 2: €320k"]

Huis → B3["📍 Buur 3: €370k"]

Huis → B4["📍 Buur 4: €315k"]

B1 & B2 & B3 & B4 → Gem["Gemiddelde ≈ €339k"]

Gem → Prijs["→ Geschatte prijs: €339k"]

KNN — Basisidee

Gelijkaardige dingen zijn dicht bij elkaar.

Algoritme:

1. Kies K = aantal buren om rekening mee te houden
2. Voor een nieuw punt:
 - Bereken de K dichtste datapunten
 - Het **meest voorkomende label** van die K punten = voorspelling

 *Training kost nauwelijks tijd — maar **classificatie is traag** (veel afstanden berekenen)*

flowchart TD

```
Start["Nieuw datapunt x"] → K["Kies K (aantal buren)"]
K → Afstand["Bereken afstand tot<br/>alle train-data punten"]
Afstand → Sorteert["Sorteer op afstand (↑)"]
Sorteert → Selecteer["Selecteer K dichtste buren"]
Selecteer → Stem["Stem: meest voorkomende<br/>label van K buren"]
Stem → Predict["Voorspel label voor x"]
```

KNN — Eigenschappen (1)

- **Supervised learning** — zowel binary als multiclass
- Werkt **enkel** als er een notie van **afstand** is tussen datapunten
- **Numerieke data**: normaliseren is **verplicht!**
 - Anders krijgen features met grote waarden onbedoeld meer gewicht
- **Categorische data**: zelf een metriek definiëren
 - Bv. vogelvluchtafstand tussen steden, of **one-hot encoding**

KNN — Eigenschappen (2)

- K moet op voorhand gekozen worden
 - Te kleine $K \rightarrow$ gevoelig voor ruis (overfitting)
 - Te grote $K \rightarrow$ te veel middeling (underfitting)
 - **Oplossing:** kies K via **cross-validatie**
- **Training:** goedkoop (enkel data opslaan) ✓
- **Classificatie:** duur (alle afstanden berekenen) ✗

*KNN is een **lazy learner** — er is geen echte trainingsfase.*

KNN — Visueel (2 kolommen)

- $k = 5$
- Het algoritme vindt **4 rood** en **1 groen** label
-  Voorspelling voor het nieuwe punt = **rood**

Implementatie met Python & sklearn

Optie 1 — Zelf implementeren (zie labo)

Optie 2 — Scikit-learn:

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
predictions = knn.predict(X_test)
```

Belangrijk!

-  Normaliseer features **vóór** KNN
-  Kies K via cross-validatie, niet "op het gevoel"
-  Gebruik `.fit` / `.predict` — dit patroon komt overal terug in sklearn